



ESTÁCIO CAMPUS MACEIÓ JATIÚCA

Curso: Desenvolvimento Full Stack

Disciplina: DGT2821 – Programação Back-end com Java

Semestre Letivo: 2026.1

Aluno : Wilton Costa

Github: <https://github.com/negoW1/DGT2821-Programa-o-Back--end-com-Java>

1. Título da Prática

Trabalho Prático – Criação de Entidades, Sistema de Persistência e Cadastro em Modo Texto com Java Orientado a Objetos.

2. Objetivo da Prática

O objetivo desta prática é desenvolver um sistema de cadastro de pessoas físicas e jurídicas utilizando os conceitos de Programação Orientada a Objetos (POO) em Java, com foco em herança, polimorfismo, encapsulamento e persistência de dados em arquivos binários.

O trabalho é dividido em dois procedimentos: o primeiro trata da criação das entidades e dos repositórios com persistência em arquivo; o segundo implementa um menu interativo em modo texto para o usuário gerenciar os cadastros.

3. Códigos Solicitados

3.1 Classe Pessoa

```
package cadastrpoo;

import java.io.Serializable;

public class Pessoa implements Serializable {
    private static final long serialVersionUID = 1L;
    private int id;
    private String nome;
```

```

public Pessoa() {}

public Pessoa(int id, String nome) {
    this.id = id;
    this.nome = nome;
}

public int getId() { return id; }
public void setId(int id) { this.id = id; }
public String getNome() { return nome; }
public void setNome(String nome) { this.nome = nome; }

public void exibir() {
    System.out.println("Id: " + id);
    System.out.println("Nome: " + nome);
}
}

```

3.2 Classe PessoaFisica

```

package cadastrpoo.model;

import java.io.Serializable;

public class PessoaFisica extends Pessoa implements Serializable {
    private static final long serialVersionUID = 2L;
    private String cpf;
    private int idade;

    public PessoaFisica() {}

    public PessoaFisica(int id, String nome, String cpf, int idade)
    {
        super(id, nome);
        this.cpf = cpf;
        this.idade = idade;
    }

    public String getCpf() { return cpf; }
    public void setCpf(String cpf) { this.cpf = cpf; }
    public int getIdade() { return idade; }
    public void setIdade(int idade) { this.idade = idade; }

    @Override
    public void exibir() {
        super.exibir();
    }
}

```

```

        System.out.println("CPF: " + cpf);
        System.out.println("Idade: " + idade);
    }
}

```

3.3 Classe PessoaJuridica

```

package cadastrpoo.model;

import java.io.Serializable;

public class PessoaJuridica extends Pessoa implements Serializable {
    private static final long serialVersionUID = 3L;
    private String cnpj;

    public PessoaJuridica() {}

    public PessoaJuridica(int id, String nome, String cnpj) {
        super(id, nome);
        this.cnpj = cnpj;
    }

    public String getCnpj() { return cnpj; }
    public void setCnpj(String cnpj) { this.cnpj = cnpj; }

    @Override
    public void exibir() {
        super.exibir();
        System.out.println("CNPJ: " + cnpj);
    }
}

```

3.4 Classe PessoaFisicaRepo

```

package cadastrpoo.model;

import java.io.*;
import java.util.ArrayList;

public class PessoaFisicaRepo {
    private ArrayList<PessoaFisica> lista = new ArrayList<>();

    public void inserir(PessoaFisica p) { lista.add(p); }

    public void alterar(PessoaFisica p) {
        for (int i = 0; i < lista.size(); i++)

```

```

        if (lista.get(i).getId() == p.getId()) { lista.set(i,
p); return; }
    }

    public void excluir(int id) { lista.removeIf(p -> p.getId() ==
id); }

    public PessoaFisica obter(int id) {
        for (PessoaFisica p : lista) if (p.getId() == id) return p;
        return null;
    }

    public ArrayList<PessoaFisica> obterTodos() { return lista; }

    public void persistir(String nomeArquivo) throws IOException {
        try (ObjectOutputStream oos = new ObjectOutputStream(
            new FileOutputStream(nomeArquivo))) {
oos.writeObject(lista); }
    }

    public void recuperar(String nomeArquivo) throws IOException,
ClassNotFoundException {
        try (ObjectInputStream ois = new ObjectInputStream(
            new FileInputStream(nomeArquivo))) {
            lista = (ArrayList<PessoaFisica>) ois.readObject(); }
    }
}

```

3.5 Classe PessoaJuridicaRepo

```

package cadastrpoo.model;

import java.io.*;
import java.util.ArrayList;

public class PessoaJuridicaRepo {
    private ArrayList<PessoaJuridica> lista = new ArrayList<>();

    public void inserir(PessoaJuridica p) { lista.add(p); }

    public void alterar(PessoaJuridica p) {
        for (int i = 0; i < lista.size(); i++)
            if (lista.get(i).getId() == p.getId()) { lista.set(i,
p); return; }
    }
}

```

```

        public void excluir(int id) { lista.removeIf(p -> p.getId() ==
id); }

        public PessoaJuridica obter(int id) {
            for (PessoaJuridica p : lista) if (p.getId() == id) return
p;
            return null;
        }

        public ArrayList<PessoaJuridica> obterTodos() { return lista; }

        public void persistir(String nomeArquivo) throws IOException {
            try (ObjectOutputStream oos = new ObjectOutputStream(
                new FileOutputStream(nomeArquivo))) {
oos.writeObject(lista); }
        }

        public void recuperar(String nomeArquivo) throws IOException,
ClassNotFoundException {
            try (ObjectInputStream ois = new ObjectInputStream(
                new FileInputStream(nomeArquivo))) {
                lista = (ArrayList<PessoaJuridica>) ois.readObject(); }
        }
    }
}

```

3.6 Classe Principal CadastroPOO (Procedimento 1)

No Procedimento 1, o método main instancia os repositórios, insere dados, persiste e recupera para demonstrar o funcionamento da serialização.

```

package cadastrpoo;
import cadastrpoo.model.*;

public class CadastroPOO {
    public static void main(String[] args) {
        try {
            PessoaFisicaRepo repo1 = new PessoaFisicaRepo();
            repo1.inserir(new PessoaFisica(1, "Ana", "11111111111",
25));
            repo1.inserir(new PessoaFisica(2, "Carlos",
"22222222222", 52));
            repo1.persistir("fisica.bin");
            System.out.println("Dados de Pessoa Fisica
Armazenados");

            PessoaFisicaRepo repo2 = new PessoaFisicaRepo();
            repo2.recuperar("fisica.bin");
        }
    }
}

```

```

        System.out.println("Dados de Pessoa Fisica
Recuperados");
        for (PessoaFisica pf : repo2.obterTodos()) pf.exibir();

        PessoaJuridicaRepo repo3 = new PessoaJuridicaRepo();
        repo3.inserir(new PessoaJuridica(3, "XPTO Sales",
"3333333333333333"));
        repo3.inserir(new PessoaJuridica(4, "XPTO Solutions",
"4444444444444444"));
        repo3.persistir("juridica.bin");
        System.out.println("Dados de Pessoa Juridica
Armazenados");

        PessoaJuridicaRepo repo4 = new PessoaJuridicaRepo();
        repo4.recuperar("juridica.bin");
        System.out.println("Dados de Pessoa Juridica
Recuperados");
        for (PessoaJuridica pj : repo4.obterTodos())
pj.exibir();
        } catch (Exception e) {
            System.out.println("Erro: " + e.getMessage());
        }
    }
}

```

3.7 Classe Principal CadastroPOO (Procedimento 2 – Menu Interativo)

No Procedimento 2, o método main é substituído por um menu interativo com Scanner, permitindo ao usuário incluir, alterar, excluir, exibir, salvar e recuperar cadastros.

```

package cadastrpoo;
import cadastrpoo.model.*;
import java.util.Scanner;

public class CadastroPOO {
    static Scanner sc = new Scanner(System.in);
    static PessoaFisicaRepo repoFisica = new PessoaFisicaRepo();
    static PessoaJuridicaRepo repoJuridica = new
PessoaJuridicaRepo();

    public static void main(String[] args) {
        int opcao;
        do {
            System.out.println("\n===== MENU =====");
            System.out.println("1 - Incluir");
            System.out.println("2 - Alterar");

```

```

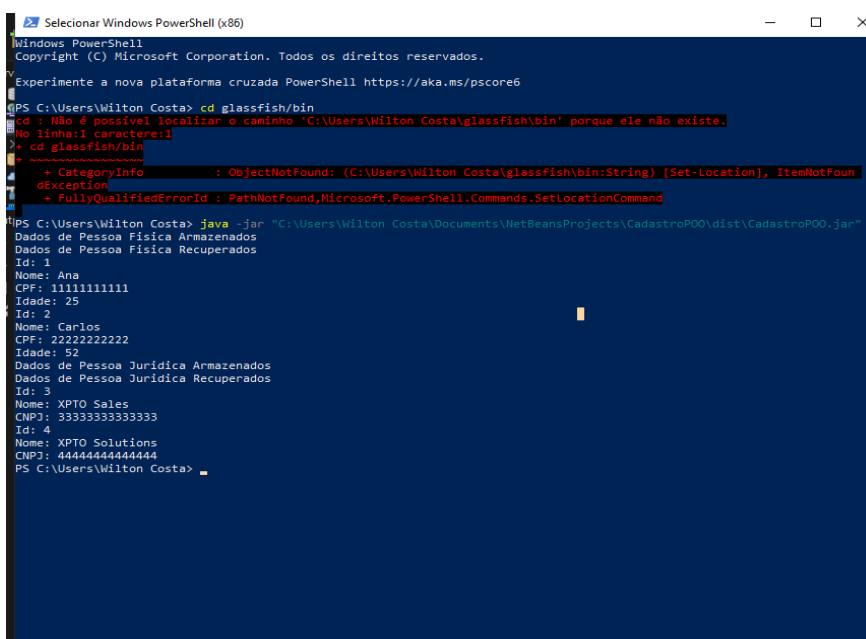
        System.out.println("3 - Excluir");
        System.out.println("4 - Exibir por ID");
        System.out.println("5 - Exibir todos");
        System.out.println("6 - Salvar dados");
        System.out.println("7 - Recuperar dados");
        System.out.println("0 - Sair");
        System.out.print("Escolha uma opcao: ");
        opcao = sc.nextInt(); sc.nextLine();
        switch (opcao) {
            case 1 -> incluir();
            case 2 -> alterar();
            case 3 -> excluir();
            case 4 -> exibirPorId();
            case 5 -> exibirTodos();
            case 6 -> salvar();
            case 7 -> recuperar();
            case 0 -> System.out.println("Encerrando...");
            default -> System.out.println("Opcao invalida!");
        }
    } while (opcao != 0);
}
// ... métodos incluir(), alterar(), excluir(), exibirPorId(),
//      exibirTodos(), salvar(), recuperar() - ver código
completo
}

```

4. Resultados da Execução

4.1 Procedimento 1 – Saída no console

Ao executar o Procedimento 1, a saída obtida foi:



```

Selecionar Windows PowerShell (x86)
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos os direitos reservados.

Experimente a nova plataforma cruzada PowerShell https://aka.ms/pscore6

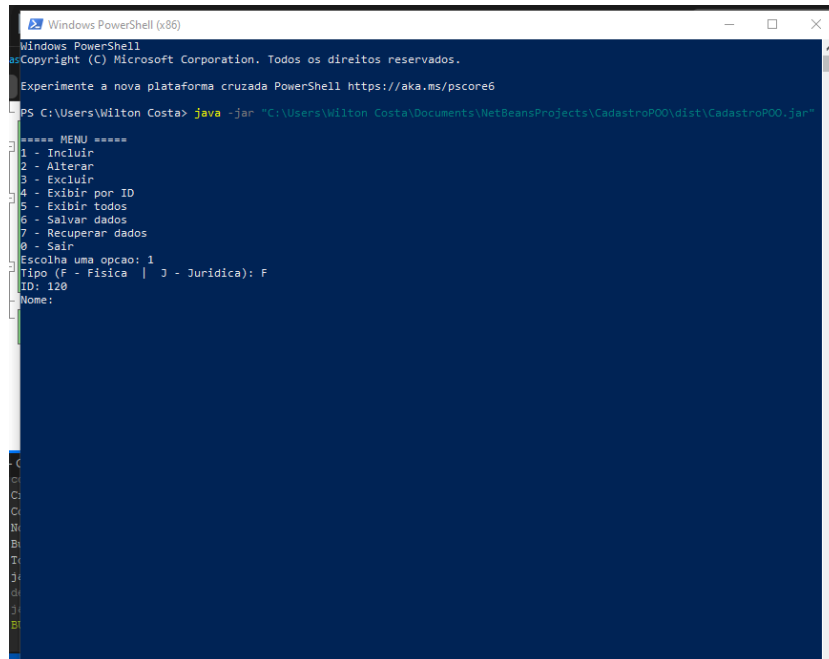
PS C:\Users\Wilton Costa> cd glassfish/bin
cd : Não é possível localizar o caminho 'C:\Users\Wilton Costa\glassfish\bin' porque ele não existe.
No linha:1 caractere:1
> cd glassfish/bin
+ ~~~~~
+ CategoryInfo          : ObjectNotFound: (C:\Users\Wilton Costa\glassfish\bin:String) [Set-Location], ItemNotFound
+ Exception             : System.IO.PathNotFoundException: O caminho não foi encontrado.
+ FullyQualifiedErrorId : PathNotFound,Microsoft.PowerShell.Commands.SetLocationCommand

PS C:\Users\Wilton Costa> java -jar "C:\Users\Wilton Costa\Documents\NetBeansProjects\CadastroPOO\dist\CadastroPOO.jar"
Dados de Pessoa Física Armazenados
Dados de Pessoa Física Recuperados
Id: 1
Nome: Ana
CPF: 11111111111
Idade: 25
Id: 2
Nome: Carlos
CPF: 22222222222
Idade: 52
Dados de Pessoa Juridica Armazenados
Dados de Pessoa Juridica Recuperados
Id: 3
Nome: XPTO Sales
CNPJ: 33333333333333
Id: 4
Nome: XPTO Solutions
CNPJ: 44444444444444
PS C:\Users\Wilton Costa>

```

4.2 Procedimento 2 – Menu interativo

O Procedimento 2 apresenta ao usuário um menu numerado. O tipo de pessoa é selecionado digitando F (Física) ou J (Jurídica). As opções de salvar e recuperar solicitam um prefixo de arquivo e tratam exceções de I/O com try-catch.



```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos os direitos reservados.
Experimente a nova plataforma cruzada PowerShell https://aka.ms/pscore6

PS C:\Users\Wilton Costa> java -jar "C:\Users\Wilton Costa\Documents\NetBeansProjects\CadastroPOO\dist\CadastroPOO.jar"

===== MENU =====
1 - Incluir
2 - Alterar
3 - Excluir
4 - Exibir por ID
5 - Exibir todos
6 - Salvar dados
7 - Recuperar dados
8 - Sair
Escolha uma opcao: 1
Tipo (F - Física | J - Juridica): F
ID: 120
Nome:
```

5. Análise e Conclusão

5.1 Vantagens e desvantagens do uso de herança

A herança é um dos pilares da Programação Orientada a Objetos e permite que uma classe (filha) herde atributos e comportamentos de outra (pai), evitando repetição de código.

Neste projeto, as classes PessoaFisica e PessoaJuridica herdam de Pessoa os campos id e nome, além do método exibir(), que é sobrescrito de forma polimórfica em cada subclasse.

Vantagens:

- Reuso de código: atributos e métodos comuns são definidos uma única vez na superclasse.

- Polimorfismo: o método `exibir()` pode ser chamado de forma genérica, com comportamento específico para cada tipo.
- Facilidade de manutenção: alterações na superclasse refletem automaticamente nas subclasses.

Desvantagens:

- Acoplamento alto: mudanças na superclasse podem impactar todas as subclasses de forma indesejada.
- Herança múltipla não é suportada em Java, o que pode limitar o design em sistemas mais complexos.
- Pode gerar hierarquias rígidas e difíceis de refatorar quando o sistema cresce.

5.2 Por que a interface `Serializable` é necessária na persistência binária?

A interface `java.io.Serializable` é um marcador (não possui métodos) que indica à JVM que os objetos de uma classe podem ser convertidos em uma sequência de bytes para serem gravados em arquivo ou transmitidos pela rede.

Sem implementar `Serializable`, ao tentar usar `ObjectOutputStream` para gravar um objeto, a JVM lança `NotSerializableException` em tempo de execução. Neste projeto, `Pessoa`, `PessoaFisica` e `PessoaJuridica` implementam `Serializable` para que os repositórios possam persistir e recuperar os `ArrayLists` corretamente usando os arquivos `.bin`.

O atributo `serialVersionUID` garante compatibilidade de versão: se a classe for alterada, objetos gravados anteriormente ainda podem ser recuperados corretamente.

5.3 Como o paradigma funcional é utilizado pela API `Stream` no Java?

A API `Stream`, introduzida no Java 8, permite processar coleções de forma declarativa e funcional, utilizando expressões `lambda` e referências de método. Em vez de descrever como iterar sobre uma lista (laço imperativo), o programador descreve o que fazer com os elementos.

Neste projeto, o método `excluir()` dos repositórios utiliza `removeIf()`, que é uma operação funcional baseada em Predicate: `lista.removeIf(p -> p.getId() == id)`. Com a API Stream completa, seria possível, por exemplo, filtrar pessoas por idade, mapear para nomes ou contar registros com operações como `filter()`, `map()` e `count()`, todas encadeadas de forma legível e concisa.

5.4 Padrão de desenvolvimento adotado na persistência de dados em Java

Na persistência com arquivos em Java, o padrão mais comumente adotado é o Repository Pattern (Padrão Repositório). Nesse padrão, a lógica de acesso a dados é isolada em classes específicas (repositórios), separando as regras de negócio do mecanismo de armazenamento.

Neste projeto, as classes `PessoaFisicaRepo` e `PessoaJuridicaRepo` encapsulam toda a lógica de inserção, alteração, exclusão, busca, persistência e recuperação. A classe principal `CadastroPOO` interage apenas com os repositórios, sem conhecer os detalhes de serialização. Isso torna o código mais organizado, testável e fácil de evoluir para outros mecanismos de persistência (banco de dados, arquivos JSON, etc.) sem alterar as entidades ou a interface do sistema.

5.5 O que são elementos estáticos e por que o método main usa static?

Elementos estáticos (modificador `static`) pertencem à classe, e não a instâncias individuais. Isso significa que podem ser acessados sem criar um objeto da classe.

O método `main` é declarado como `static` porque a JVM precisa chamá-lo antes de qualquer objeto ser criado. Se `main` não fosse estático, seria necessário instanciar a classe antes de executá-la, o que criaria uma dependência circular. Neste projeto, os atributos `Scanner`, `PessoaFisicaRepo` e `PessoaJuridicaRepo` também foram declarados `static` para serem compartilhados entre todos os métodos estáticos da classe principal.

5.6 Para que serve a classe Scanner?

A classe `java.util.Scanner` é utilizada para leitura de dados a partir de diversas fontes, como teclado (`System.in`), arquivos ou strings. Ela interpreta a entrada e disponibiliza métodos como `nextInt()`, `nextLine()` e `nextDouble()` para capturar valores em diferentes tipos.

No Procedimento 2, o `Scanner` é responsável por receber todas as entradas do usuário: opções do menu, tipo de pessoa, ID, nome, CPF/CNPJ, idade e prefixo de arquivo. O cuidado com `sc.nextLine()` após `sc.nextInt()` é necessário para consumir o caractere de quebra de linha que fica no buffer após a leitura de um número.

5.7 Como o uso de classes de repositório impactou a organização do código?

A separação em classes de repositório (`PessoaFisicaRepo` e `PessoaJuridicaRepo`) trouxe benefícios claros de organização. A classe principal ficou responsável apenas pela interface com o usuário (menu, leitura de dados), enquanto toda a lógica de gerenciamento e persistência ficou encapsulada nos repositórios.

Isso respeita o princípio da responsabilidade única (Single Responsibility Principle), um dos princípios SOLID. O código ficou mais legível, mais fácil de testar individualmente e mais simples de manter. Futuramente, se o mecanismo de armazenamento mudar (ex: banco de dados), apenas os repositórios precisariam ser alterados, sem impactar o restante do sistema.